

ICS-NAD Data Quality & Preprocessing Report

Phase 1 (Data Engineering) findings — MS thesis: ICS-NAD-Gym + LLM-shaped RL for ICS intrusion detection

Generated 2026-07-08 · Source dataset: ICS-NAD (Scientific Data / Nature, 2026), `Labeled_CSV` subset, 61 files, 19.2 GB
· Pipeline: `src/icsnad_gym/`

Why this report exists. Before any RL environment or supervised baseline can be trusted, the ground-truth labels they train against have to be trustworthy. This report documents a systematic audit of all 61 raw capture files, the label-quality issues found (more numerous than initially estimated), how each was resolved, and the resulting train/validation/test split and feature-normalization statistics used downstream.

1. Executive Summary

61 files audited	60,132,332 total flow rows	15 / 61 files with corrected labels	13,880,552 rows flagged label_uncertain
45 clean, trusted as-is	7 wrong label string, renamed	8 row-level reconstructed	1 excluded (too small)

The dataset's published `state` labels were generated by the original authors' per-file, copy-pasted labeling scripts (`external/Code-for-ICS-NAD-Dataset/labeling_tcx.py` , `labeling_ws.py`) that hardcode a victim IP and attack-name string per file. These scripts were not always updated correctly when reused, producing three categories of defect found by direct inspection rather than assumed from the paper:

- Wrong attack-type string, correct row split** — the BENIGN/attack boundary is right, but the attack rows carry the wrong name (e.g. a synflood capture entirely labeled `ACKFLOOD`).
- Attack traffic silently labeled BENIGN** — the labeling script found nothing to flag at all; millions of real attack rows carry the default label.
- No `state` column present** — five files ship with only the 60 feature columns, no label at all.

2. Corrected Files — Full Detail

Every row whose label was overridden or reconstructed is flagged `label_uncertain=True` in the harmonized Parquet output, keeping the correction fully auditable downstream.

File	Method	Canonical label	Rows	Rows corrected	Original label(s)
ABB_LONG_synflood_faster_1_01-.csv	rename	SYNFLOOD	4,999,765	4,999,021	ACKFLOOD

ABB_LONG_tcpflood_faster_1_01-.csv	rename	TCPFLOOD	4,999,658	2,792,417	ACKFLOOD, TCPFLOOD (mixed)
ABB_teardrop_fast_0715.csv	rename	TEARDROP	9,017	2,402	ICMPFLOOD
Schneider_tcpall0_0710.csv	rename	TCPALL	8,150	2,433	TCP (generic placeholder)
Schneider_tcpfin1_0710.csv	rename	TCPFIN1	7,932	2,392	TCP (generic placeholder)
Schneider_tcpflood_fast_0710.csv	rename	TCPFLOOD	3,133	2,406	TCP (generic placeholder)
Schneider_tcpsynfin0_0710.csv	rename	TCPSYNFIN	7,923	2,373	TCP (generic placeholder)
ABB_land_faster_0715_part1.csv	signature	LAND	2,787	395	– (no state column)
Siemens_tcpall0_0709.csv	signature	TCPALL	4,844	804	– (no state column)
Siemens_tcpfin1_0709.csv	signature	TCPFIN1	4,807	969	– (no state column)
Siemens_tcpsynfin0_0709.csv	signature	TCPSYNFIN	4,596	803	– (no state column)
Siemens_teardrop_0709.csv	signature	TEARDROP	4,803	2,176	– (no state column)
Schneider_tcpflood_faster_0723_part1.csv	signature	TCPFLOOD	4,978,791	2,090,636	BENIGN (100% – attack entirely unflagged)
Siemens_LONG_synflood_fast_1_01-.csv	signature	SYNFLOOD	4,997,671	2,029,304	BENIGN (100% – attack entirely unflagged)
Siemens_LONG_tcpflood_fast_1_01(1).csv	signature	TCPFLOOD	4,997,571	1,952,021	BENIGN (100% – attack entirely unflagged)
Schneider_nmap_0710-.csv	exclude	NMAP	374	0	BENIGN – too small to reconstruct reliably

2.1 Reconstruction method

Rename (7 files): the BENIGN/attack row split in the original `state` column is trustworthy — only the attack-type string was wrong. Fixed by overriding non-BENIGN rows to the filename-derived canonical label.

Signature (8 files): row-level split itself was missing or wrong, so it was reconstructed from validated protocol-level features — `sAddress==rAddress` for LAND, fragment rate for TEARDROP, TCP flag-rate combinations for the TCPFIN1/TCPALL/TCPSYNFIN family, and SYN-rate/ACK-rate profile for SYNFLOOD/TCPFLOOD. Each signature was validated against a known-clean reference file before use

(e.g. TCPFIN1's `sFinRate=1, sSynRate=0` profile cleanly separates 2,385 attack rows from 6,497 benign rows with zero overlap in `ABB_tcpfin1_0715-.csv`).

Exclude (1 file): `Schneider_nmap_0710-.csv` is only 374 rows, and the NMAP addressing scheme observed in a reference file (broadcast-IP based) did not transfer to this file's traffic pattern — reconstructing it would be guessing. Left as-is and flagged in this report rather than forcing an unreliable fix.

Known limitation — burst-count corroboration. Each reconstructed file's non-BENIGN row count was cross-checked against the number of attack trials logged in the corresponding `Time_of_Attack` file (19 trials per capture). For the three highest-volume signature-reconstructed files this check produced burst counts in the hundreds of thousands to over a million, vastly exceeding the 19 logged trials. Direct inspection confirmed this is *not* a counting bug: during an active flood, legitimate background TCP traffic genuinely interleaves row-by-row with attack flows at the flow-record level, so the signature correctly flags individual attack flows but the result doesn't collapse into 19 clean contiguous bursts. Per-flow signature accuracy remains validated against reference files; simple burst-counting is documented here as an unreliable corroboration metric for these three files specifically, not a validation failure.

3. Raw Data Corruption (Independent of Labeling)

Two files had genuine, unrelated data corruption from what appears to be an interrupted write during the original capture-to-CSV export — found via a full raw comma-count scan of every line, not inferred from a parser error message alone (which initially reported misleading byte offsets).

File	Defect	Scale	Resolution
ABB_LONG_ackflood_faster_1_01-.csv	Line 1,284,059 truncated mid-write (MAC address cut off, e.g. <code>90:8d:6...</code>); missing newline caused a 30-field row instead of 61	1 row of 4,993,599 (0.00002%)	Dropped via <code>ignore_errors=True</code> ; row count after cleaning matches an independent raw comma-count scan exactly
ABB_LONG_icmpflood_faster_1_01-.csv	Row(s) with more fields than the 61-column schema	Negligible	Handled via <code>truncate_ragged_lines=True</code>

Both fixes are applied generically in `src/icsnad_gym/schema.py` to every file read, not just these two, in case similar isolated corruption exists elsewhere undetected.

4. Train / Validation / Test Split

Policy: file-level holdout wherever an attack type has multiple capture files (prevents leaking the same session across train/eval); time-ordered intra-file split only for attack types existing in exactly one file. Of 20 canonical attack types, only **NMAP** and **ICMPREDIRECT** are single-file and use the intra-file fallback (70/15/15 by time, with buffer gaps between segments).

Bug found and fixed during this phase. The first split implementation sorted each attack type's files alphabetically before assigning train/val/test roles. Because "Siemens" sorts after "ABB" and "Schneider", Siemens files were *systematically* assigned to val/test and never to train, for every multi-brand attack type — undermining the brand-diversity premise the whole environment design depends on. Fixed by switching to a deterministic hash-based file ordering, decorrelating split assignment from brand name.

4.1 File-level split counts

Split	Files
train	33
val	8
test	18

4.2 Brand representation (after the fix)

Brand	Train files	Val files	Test files	Train rows
-------	-------------	-----------	------------	------------

ABB	8	0	9	5,045,483
Schneider	15	6	5	67,098
Siemens	10	2	4	15,013,049

All three brands now have non-zero train representation. ABB has zero val files by chance of the hash assignment across its specific label set — every ABB attack type still has val or test coverage from a sibling brand's file, so this does not remove any class from evaluation entirely.

4.3 Per-attack-type validation coverage gap (expected, not a bug)

Attack types with only 2–3 total capture files necessarily get train+test only, no independent validation file, under the "at most 2 files held out" rule. Affected: LAND, PORTSCAN, ROUTEIP, ARP, ICMPNOTREACH, ICMPBIG, TCPALL, TCPFIN1, TEARDROP. This is a direct consequence of real data scarcity (2–3 sessions were captured for these attack types in the source dataset) and is documented here rather than silently accepted — model selection for these classes will rely on aggregate/other-class validation signal, not a class-specific held-out set.

5. Feature Normalization (Per-Brand Scalers)

Z-score scalers (mean, std) were fit independently per brand, on **train-split rows only**, over the 49 numeric flow features used as the RL/supervised observation space (identity fields — IPs, MACs, absolute timestamps — are excluded by design so a policy can't shortcut by memorizing fixed addresses; protocol is one-hot encoded separately, not z-scored).

Brand	Train rows used	Features scaled	Sanity check
ABB	5,048,770	49	no NaN / Inf / zero-std
Schneider	67,642	49	no NaN / Inf / zero-std
Siemens	15,013,049	49	no NaN / Inf / zero-std

Constant-valued columns (std $\approx$ 0) are guarded with a std floor of 1.0 to avoid division-by-zero rather than silently producing NaN/Inf downstream.

6. Pipeline & Reproducibility

All logic described in this report is implemented in `src/icsnad_gym/` :

Module	Purpose
<code>schema.py</code>	Explicit 61-column dtype schema; robust CSV reading (<code>ignore_errors</code> , <code>truncate_ragged_lines</code>)
<code>label_harmonizer.py</code>	Canonical label mapping, filename→expected-label dispatch, per-file strategy table, signature reconstruction
<code>time_of_attack.py</code>	<code>Time_of_Attack</code> log parsing and burst-count corroboration check
<code>build_dataset.py</code>	Batch driver: harmonize all 61 files, write Parquet, emit <code>label_quality_report.json</code>
<code>splits.py</code>	Train/val/test assignment (file-level + intra-file fallback), emits <code>splits.json</code>
<code>scalers.py</code>	Per-brand z-score fitting on train split only, emits <code>scalers.json</code>

Outputs: `data/parquet/` (61 harmonized files, 2.3 GB, down from 19.2 GB CSV),
`data/label_quality_report.json` , `data/splits.json` , `data/scalers.json` .

7. Next Steps

This closes Phase 1 of the approved thesis plan. Phase 2 builds `ICSNADGym` itself (Gymnasium environment, quarantine state machine, base reward table) and runs the mandatory sanity-floor checks (random / always-ALLOW / always-ALERT-BLOCK / oracle policies) before any RL training is trusted.